

PneumoNet

*Attention-Guided and Explainable Deep Learning for Robust
Multi-Class Chest X-Ray Classification*

Project Report

Final-Year Computer Engineering / Data Science Project

Sections: Introduction · System Architecture · Workflow · Gantt Chart · Hardware/Software Specifications · Sequence Diagram ·
DFD · Class Diagram · UML Use Case Diagram · Week-wise Implementation Plan · References

1. Introduction

1.1 Background

Chest radiography (CXR) is one of the most widely used and cost-effective medical imaging modalities for screening and diagnosing thoracic diseases. With the growing availability of large public chest X-ray datasets and advances in deep learning, automated CXR classification has become an active research area combining computer vision, medical imaging, and explainable AI (XAI).

PneumoNet is a research-oriented deep learning system designed to classify chest X-rays into four categories — Normal, Pneumonia, Tuberculosis (TB), and COVID-19 — while investigating whether attention mechanisms genuinely improve classification performance, whether that improvement generalizes across datasets, and whether model predictions can be explained in a way that is anatomically meaningful (via Grad-CAM).

1.2 Motivation

- Several thoracic diseases exhibit overlapping radiographic patterns, making manual and automated diagnosis challenging.
- A classifier can perform well in aggregate while quietly failing on a specific disease class — aggregate accuracy alone is not trustworthy in a medical context.
- Public datasets differ significantly in acquisition conditions, patient populations, labeling methodology, and class balance.
- Deep learning predictions are difficult to interpret directly; Grad-CAM offers a coarse but useful visual explanation of model attention.
- A model that performs well internally may fail to generalize externally — this must be tested, not assumed.

1.3 Problem Statement

Develop and experimentally validate a reproducible deep-learning framework capable of classifying chest X-ray images into Normal, Pneumonia, Tuberculosis, and COVID-19, while systematically investigating the effect of CNN architecture and attention mechanisms, analyzing prediction explanations with Grad-CAM, and assessing whether observed improvements generalize beyond the development distribution — and then productionizing the validated model as a usable, end-to-end web application.

1.4 Objectives

- Review CXR deep-learning, transfer-learning, attention, and explainability literature.
- Acquire, harmonize, and audit three public datasets (NIH ChestX-ray14, TBX11K, Curated COVID-19).
- Build a reproducible preprocessing and label-harmonization pipeline.
- Train CNN baselines (ResNet, DenseNet, EfficientNet) and augment them with attention (SE, spatial, CBAM).
- Perform ablation studies and evaluate class-wise performance using appropriate imbalance-aware metrics.
- Use Grad-CAM to analyze and, where possible, quantitatively validate prediction attribution.
- Evaluate internal vs. external/cross-dataset generalization.
- Track experiments with MLflow and deploy the validated model as a full-stack web product with authentication, history, and report generation.

1.5 Scope and Limitations

The system is a research and decision-support prototype, not a certified diagnostic device. It does not replace radiologist interpretation, is not clinically validated, and should be positioned as an educational/decision-support tool. The four classes are treated as mutually exclusive, which is a simplification of real-world thoracic diagnosis where co-occurring findings are common.

2. System Architecture

The system follows a layered architecture separating the client, application, data, and infrastructure concerns, allowing the research model and the production product to evolve independently while sharing the same trained artifact via the model registry.

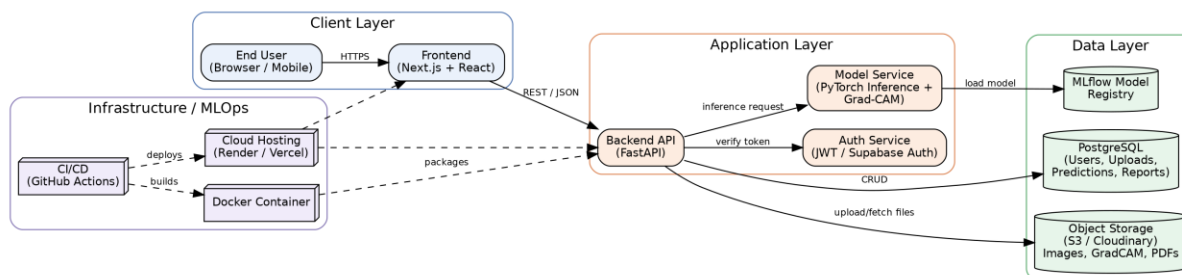


Figure 2.1 — PneumoNet System Architecture

2.1 Layer Descriptions

- **Client Layer:** A Next.js/React frontend that end users interact with directly in the browser.
- **Application Layer:** A FastAPI backend exposing REST endpoints, an authentication service (JWT-based or Supabase Auth), and a model service that wraps the trained PyTorch model plus Grad-CAM generation.
- **Data Layer:** PostgreSQL for structured data (users, uploads, predictions, reports), object storage (S3/Cloudinary) for images/heatmaps/PDFs, and an MLflow Model Registry holding versioned, validated models.
- **Infrastructure/MLOps Layer:** Docker containerizes the inference service; GitHub Actions runs tests and deployments (CI/CD); the app is hosted on Render/Vercel or an equivalent cloud platform.

3. Workflow

The end-to-end workflow spans the research pipeline (left/top of the diagram) and the production pipeline (bottom), converging at the MLflow Model Registry, which is the single handoff point between research and product.

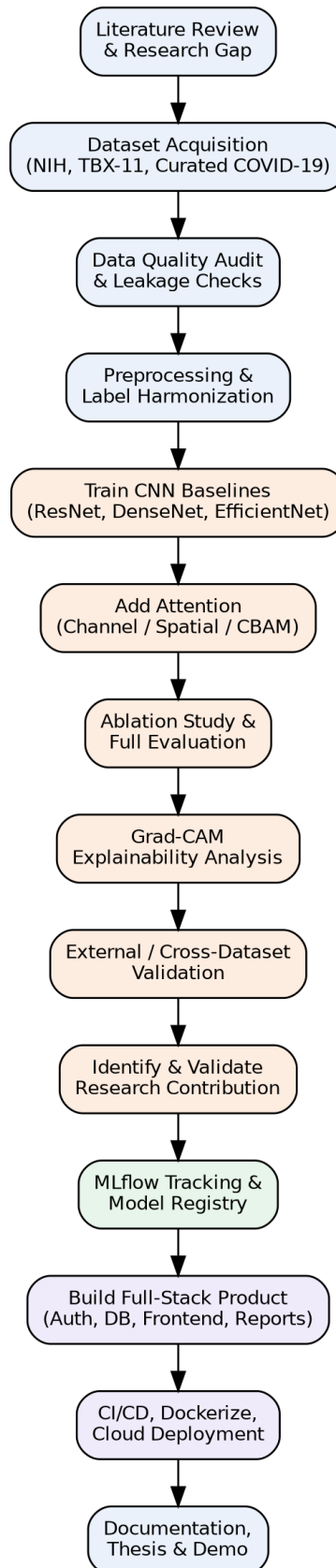


Figure 3.1 — End-to-End Project Workflow

4. Gantt Chart

The project is planned across a two-semester, ~31-week timeline. Semester 1 focuses on the research/model track (foundations through validated research contribution); Semester 2 focuses on explainability finalization, the full-stack product build, and deployment/documentation.

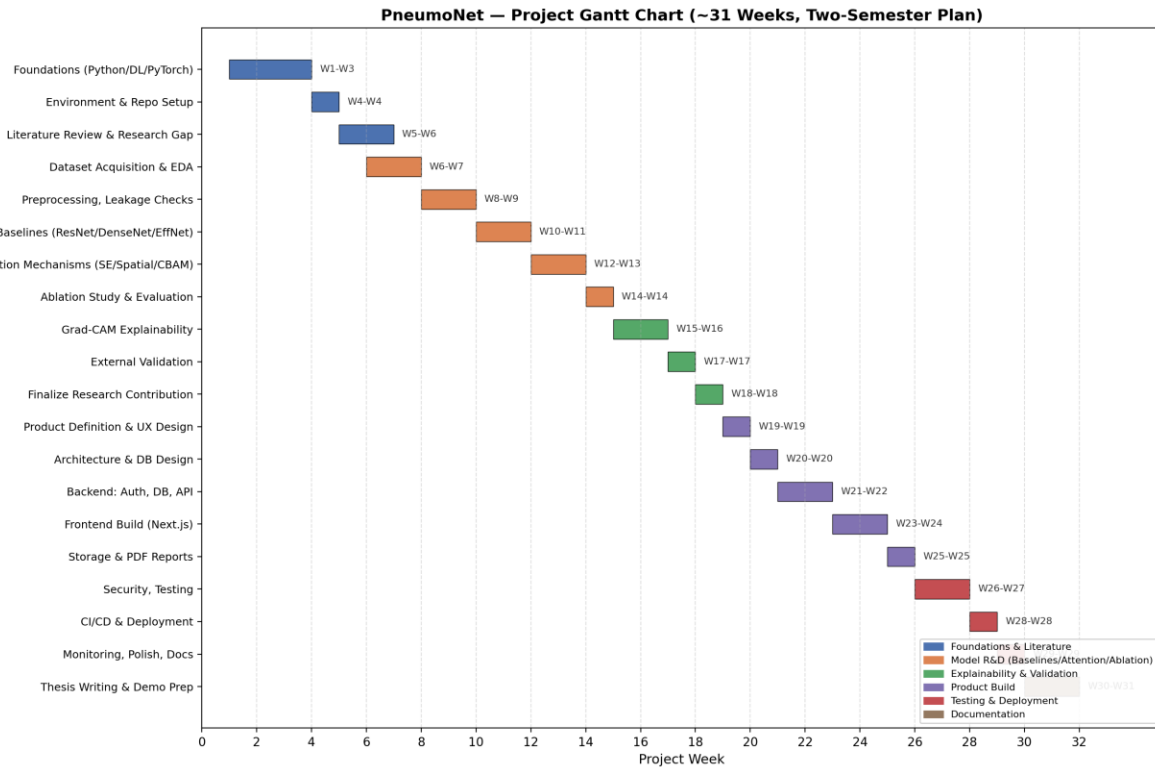


Figure 4.1 — Project Gantt Chart

5. Hardware / Software Specifications

5.1 Development Hardware

Component	Minimum	Recommended
Processor (CPU)	Quad-core (Intel i5 / Ryzen 5)	8-core (Intel i7/i9, Ryzen 7/9)
RAM	8 GB	16–32 GB
GPU (local, optional)	None (use cloud GPU)	NVIDIA RTX 3060+ (8GB+ VRAM)
Cloud GPU (training)	Kaggle T4 (free, 30 hrs/week)	Colab Pro / A100 (paid) for faster iteration
Storage	50 GB free (datasets + envs)	150+ GB SSD
Internet	Stable broadband	High-speed (large dataset downloads)

5.2 Production/Deployment Hardware

Service	Specification
Backend inference server	2 vCPU / 4GB RAM minimum (CPU inference); GPU instance optional for high throughput
Database	Managed PostgreSQL, 1–2 GB storage to start (Supabase/Neon free tier sufficient for a prototype)
Object storage	5–20 GB for images/heatmaps/PDFs to start (Cloudinary/S3 free tier)

5.3 Software Stack

Category	Technology
Programming language	Python 3.10+, JavaScript/TypeScript (frontend)
DL framework	PyTorch, torchvision / timm
Data handling	Pandas, NumPy, OpenCV, PIL, Albumentations
Experiment tracking	MLflow (tracking + model registry)
Explainability	pytorch-grad-cam
Backend framework	FastAPI, Uvicorn, SQLAlchemy, Alembic
Frontend framework	Next.js, React, Tailwind CSS
Database	PostgreSQL (Supabase / Neon)
Authentication	JWT or Supabase Auth
File storage	AWS S3 or Cloudinary
PDF generation	WeasyPrint / ReportLab
Testing	Pytest, Playwright/Cypress
Containerization	Docker
CI/CD	GitHub Actions
Hosting	Render/Railway (backend), Vercel (frontend)
Monitoring	Sentry (errors), PostHog/GA (analytics)
Version control	Git + GitHub

6. Sequence Diagram

The sequence diagram below illustrates the primary user interaction: uploading a chest X-ray, receiving a classification with a Grad-CAM explanation, and downloading a PDF report.

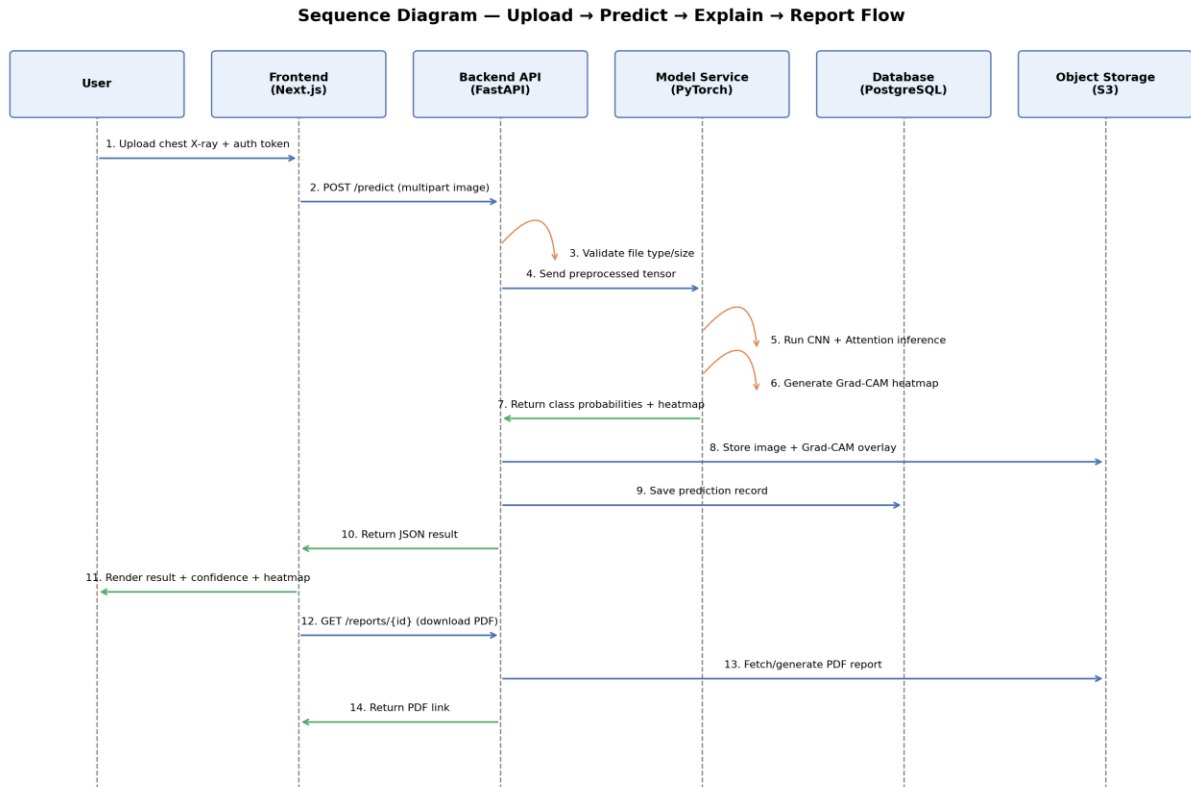


Figure 6.1 — Sequence Diagram: Upload → Predict → Explain → Report

7. Data Flow Diagrams (DFD)

7.1 DFD Level 0 (Context Diagram)

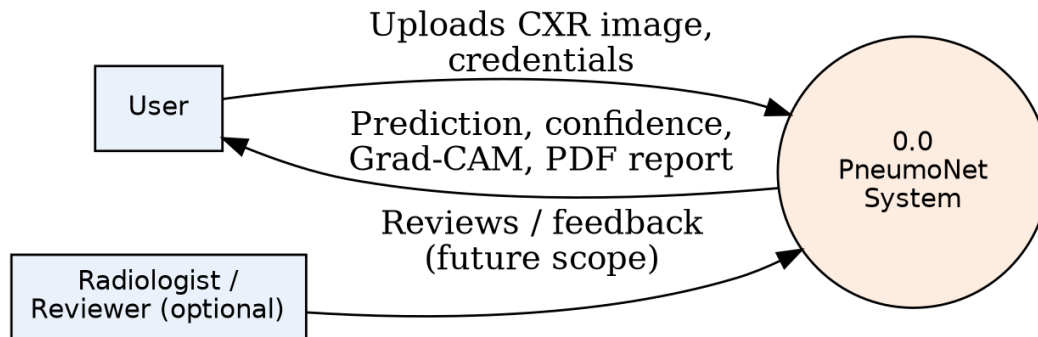


Figure 7.1 — DFD Level 0 (Context Diagram)

7.2 DFD Level 1

Level 1 decomposes the system into its six core processes: authentication, image validation/preprocessing, inference, Grad-CAM generation, report generation, and history storage/retrieval, along with the four data stores they interact with.

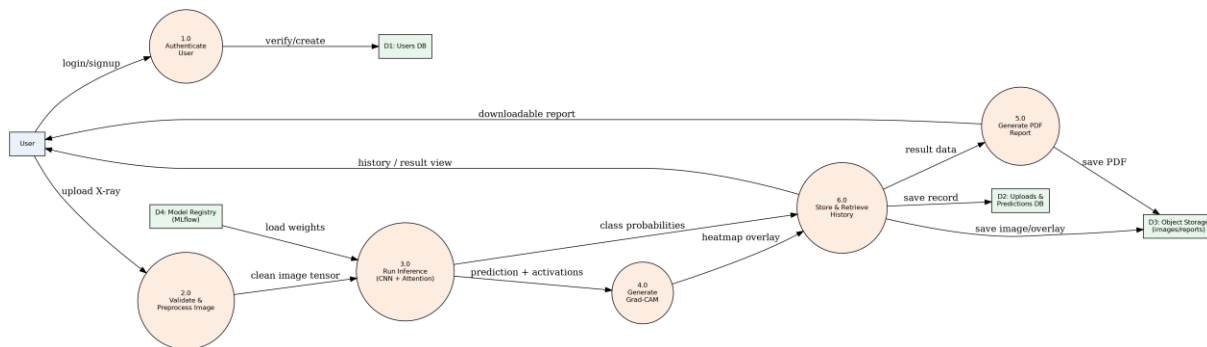


Figure 7.2 — DFD Level 1

8. Class Diagram

The core domain model centers on Users producing Uploads, each Upload yielding one Prediction, which may generate a Report. The ModelService composes a CNNBackbone with a pluggable AttentionModule, reflecting the controlled-experiment design of the research track.

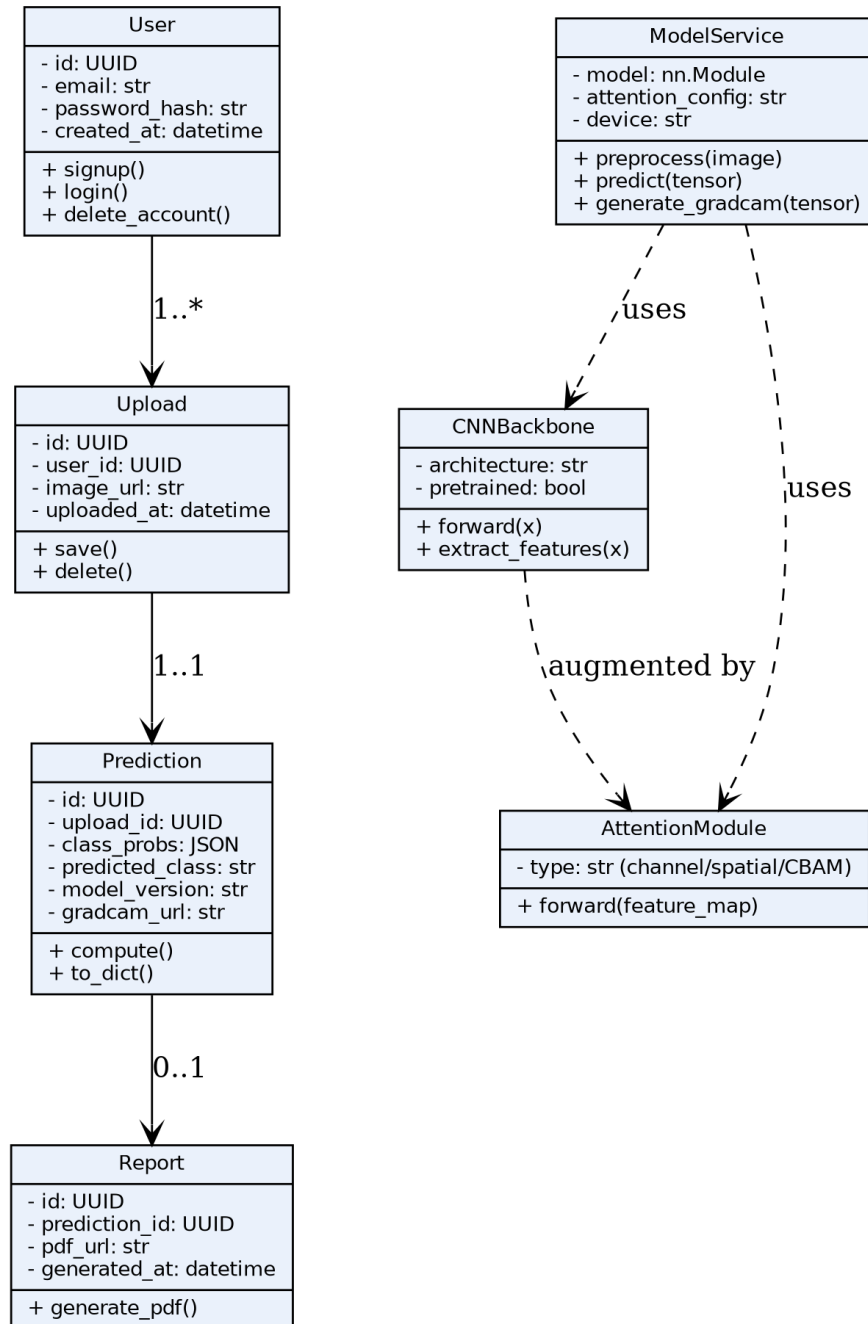


Figure 8.1 — Class Diagram

9. UML Diagram — Use Case Diagram

Two actors interact with the system: the End User (uploads scans, views results/history, manages their account) and the Admin/Researcher (registers new models, monitors system health).

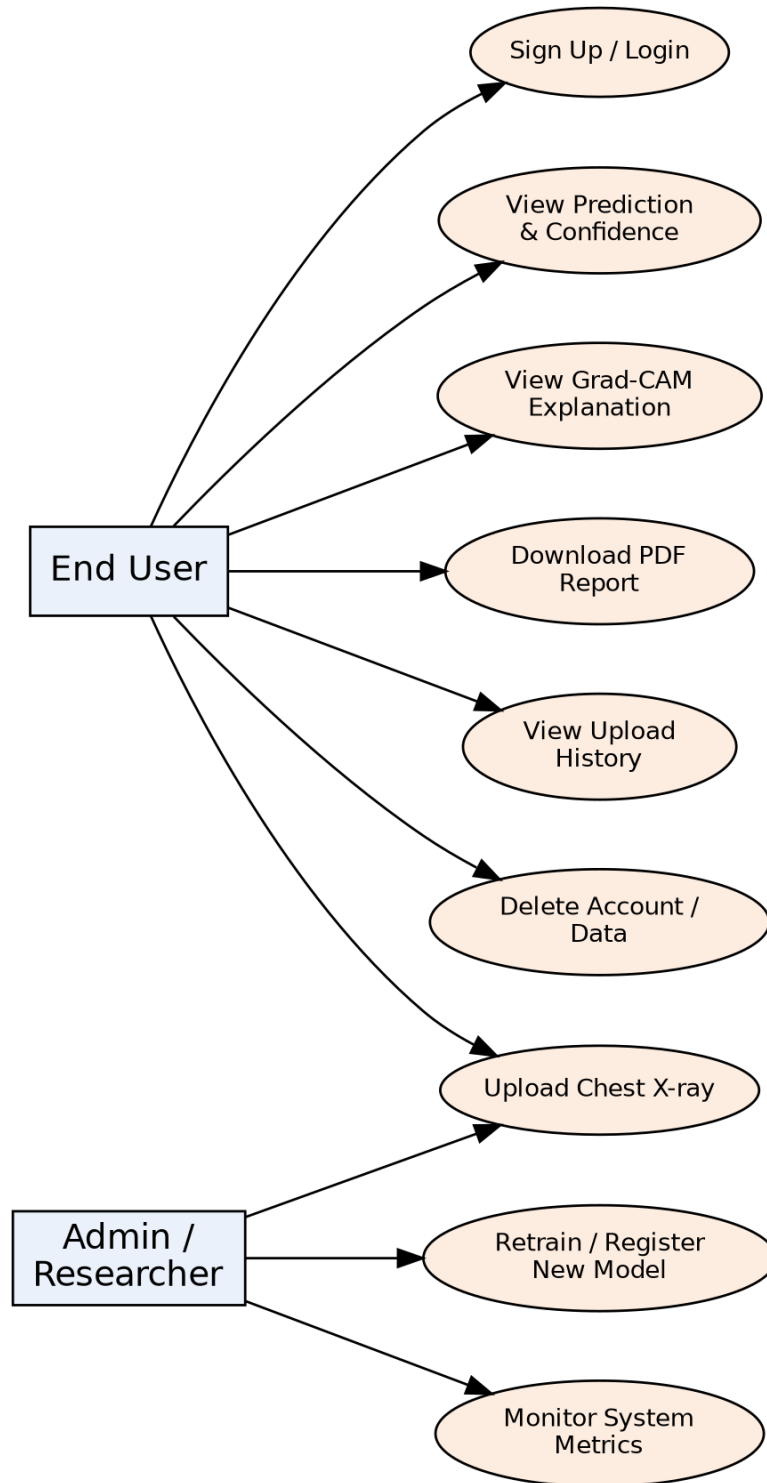


Figure 9.1 — UML Use Case Diagram

10. Implementation Plan (Week-wise)

Semester 1 — Research & Model Track

Week	Task	Deliverable
1–3	Python, ML/DL, and PyTorch foundations	Working toy CNN classifier
4	Environment & repository setup	Repo skeleton, MLflow smoke test
5–6	Literature review & research gap definition	Literature review + research-gap document
6–7	Dataset acquisition & EDA (NIH, TBX11K, Curated COVID-19)	EDA notebooks + label-harmonization scheme
8–9	Preprocessing pipeline, leakage checks, patient-level split	Reusable preprocessing module + dataset manifest
10–11	Train CNN baselines (ResNet, DenseNet, EfficientNet)	3 trained baselines + MLflow logs
12–13	Implement & train attention variants (SE, spatial, CBAM)	Attention-augmented models + comparison table
14	Ablation study & full evaluation	Ablation results + analysis

Semester 2 — Explainability, Contribution & Product Track

Week	Task	Deliverable
15–16	Grad-CAM explainability + quantitative validation (TBX11K boxes)	Grad-CAM comparison + IoU/localization score
17	External/cross-dataset validation	Internal vs. external comparison report
18	Finalize & validate research contribution	Validated contribution with before/after evidence
19	Product definition & UX design	Wireframes + user-flow diagram
20	System architecture & database schema design	ERD + hosting decisions
21–22	Backend: auth, database, extended product API	Authenticated API covering full user flow
23–24	Frontend build (Next.js): upload, result, history, reports	Full working frontend against real backend
25	File storage integration & PDF report generation	Downloadable PDF reports
26–27	Security/privacy pass + testing (unit, integration, e2e)	Test suite + legal/privacy pages
28	CI/CD pipeline + full-stack cloud deployment	Live, publicly reachable product
29	Monitoring, analytics, UI polish	Monitored, polished product
30–31	Documentation, thesis writing, demo video, viva prep	Final report, slides, demo video

11. References

[1] X. Wang, Y. Peng, L. Lu, Z. Lu, M. Bagheri, and R. M. Summers, "ChestX-ray8: Hospital-Scale Chest X-Ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases," in Proc. IEEE CVPR, 2017.

- [2] Y. Liu, Y.-H. Wu, Y. Ban, H. Wang, and M.-M. Cheng, "Rethinking Computer-Aided Tuberculosis Diagnosis," in Proc. IEEE/CVF CVPR, 2020. (TBX11K dataset)
- [3] U. Sait et al., "Curated Dataset for COVID-19 Posterior-Anterior Chest Radiography Images (X-Rays)," Mendeley Data, 2020.
- [4] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in Proc. IEEE ICCV, 2017.
- [5] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in Proc. IEEE/CVF CVPR, 2018.
- [6] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional Block Attention Module," in Proc. ECCV, 2018.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, 2016. (ResNet)
- [8] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," in Proc. IEEE CVPR, 2017. (DenseNet)
- [9] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. ICML, 2019.
- [10] E. Çallı, E. Sogancioglu, B. van Ginneken, K. G. van Leeuwen, and K. Murphy, "Deep learning for chest X-ray analysis: A survey," *Medical Image Analysis*, vol. 72, 2021.
- [11] Y. Chen et al., "Chest X-ray analysis empowered with deep learning: A systematic review," *Applied Soft Computing*, 2022.
- [12] Explanatory classification study of chest X-ray images into COVID-19, Pneumonia, and Tuberculosis using XAI (Grad-CAM, LIME, SHAP), 2022.
- [13] Systematic review: "External Validation of Deep Learning Algorithms for Radiologic Diagnosis," (reports reduced performance on external datasets in the majority of evaluated studies).
- [14] PyTorch documentation, <https://pytorch.org/docs/>
- [15] FastAPI documentation, <https://fastapi.tiangolo.com/>
- [16] MLflow documentation, <https://mlflow.org/docs/latest/index.html>
- [17] NIH Clinical Center, "NIH Chest X-ray Dataset," <https://nihcc.app.box.com/v/ChestXray-NIHCC>